

Component Software Development: An Introduction

Summary

Fashions come and go in all industries, and the latest trend to hit computing is component-based development. Many pundits as well as less erudite users tend to confuse components with objects, but the two constructs are very different. Although they share the goal of reusing existing software, they do it in different ways: objects offer reuse through inheritance, whereas components provide reuse through encapsulation. Components appear to be much more successful at reaching this goal than objects, and the reasons are identified below.

Table of Contents

- Technology Basics
- What Is a Component?
- Technology Analysis
- Business Use
- Benefits and Risks
- Standards
- Selection Guidelines
- Technology Leaders
- Datapro Insight

List Of Tables

- Table 1: Component Standards

Component Software Development: An Introduction

Technology Basics

What Is a Component?

Put very simply, a component is an independent software unit with a well-defined and documented set of interfaces for providing and using services and connecting to other software. It may be written in any language— object-oriented or not— and may also offer executable code, testing procedures, additional plug-ins, and default properties.

A component makes itself known to its clients through one or more public interfaces. These interfaces encapsulate the component's behavior and hide the actual implementation of the behavior .The clients only need to know that the behavior exists, but not the specifics of its implementation. The behavior— or even the component itself— can be changed without the client's knowledge, as long as the relevant interface is constant.

The component's container is also critical to its behavior. In many component models— especially Enterprise JavaBeans (EJBs)— the container takes care of messaging, event handling, and overall properties. EJBs also derive properties from their containers, and many application server and middleware vendors supply EJB container specifically designed for their products.

Well-designed components, therefore, help applications evolve and stay current by providing pluggability and extensibility. Components let developers plug in new features, remove others, and modify still others without affecting the rest of the application.

Components can be designed primarily to work together or function independently. The tighter the coupling, the easier it is for components to work together and provide rapid processing. This does, however, impede reusability and maintenance. Loosely coupled components generally exchange data rather than process logic and are easier to reuse and maintain, but they often require translation and messaging software— as well as common data formats— to make communication possible.

Components and Objects: Differences

Unfortunately, much of the world uses "components" and "objects" to mean the same thing— a relatively small, binary unit of software. In reality, the two concepts are fundamentally very different, even though they do share some high-level behavior.

Definitions

Objects are abstractions specified through a series of definitions; as such, they are not directly manipulated by applications. What appears in the software are instances— or realizations— of these abstractions.

Components, on the other hand, are real pieces of code that provide services to and require services from applications or other components. Each component behaves— and is managed— as a single unit with respect to the software that uses it.

Use

Objects encapsulate data and operations. They help developers discover real-world entities and model their relationships and associated events. Objects are most applied in the analysis and design phases of an application's life cycle and tend to provide very fine-grained functionality.

Component Software Development: An Introduction

Components also provide encapsulation, but with much larger granularity. What is encapsulated can include objects, other components, non-object-oriented code, and even entire applications. Components let developers add, modify, or delete services during the implementation or maintenance phases without disrupting the larger application. When designed properly, components can be used essentially unchanged in a variety of environments.

Reuse

In most object-oriented systems, the objects are arranged in hierarchies of classes. Classes at the lower levels of the hierarchy inherit attributes and methods from the parent classes above them. Therefore, reuse depends primarily upon inheritance and secondarily on encapsulation. Unfortunately, this tends to limit information hiding, since object users must understand how methods and attributes inherited from parent classes are implemented in child classes.

Components are completely independent from one another without hierarchical dependencies, so there are no inheritance trees to worry about. Components use encapsulation to hide how their behavior is implemented so that clients do not need to know the particulars of how the behavior is implemented. Ideally, different components that share the same interface should be interchangeable, and a single component with a well-defined interface should be able to be plugged into multiple environments. Therefore, component reusability depends almost entirely on encapsulation.

Analysis and Design Tools

Tools and methodologies for designing objects have been available for nearly 20 years. Until the late 1990s, design tools and notations proliferated to the point where there were dozens of ways to create object-oriented systems and design class hierarchies. Unfortunately, this frightened away many organizations and developers, since it was difficult at best to transition from one product or methodology—and its notation—to another. The popularity of the UML is likely to make object-oriented analysis and design much easier by standardizing notation across both methodologies and products.

On the other hand, there are no tools or methodologies dedicated to component development. The UML has some support for components, but this is still under development. Brian Henderson-Sellers' OPEN and Desmond D'Souza's Catalysis methodologies provide some analysis and design features, and Rational's Visual Modeler supports Microsoft components.

Component Models

Today two component models dominate the market: Microsoft's Component Object Model (COM) and Sun Microsystems' JavaBeans and Enterprise JavaBeans (EJB). A third model—the Object Management Group's CORBA Components—appeared in late 1999.

Microsoft Common Object Model (COM)

Microsoft's COM is a component-only model that emphasizes interoperability and packaging. It supports encapsulation and polymorphism but not inheritance, although developers can use object-oriented languages such as C++ or Java to create COM components and interfaces. COM evolved from two Windows technologies: dynamic data exchange (DDE)—which let users hot-link information between two applications—and object linking and embedding (OLE)—a technology for creating compound documents with multiple applications. The first version of COM was a faster OLE with encapsulation.

In 1993 Visual Basic popularized the notion of components with the VBX controls, which let users add features, first to Visual Basic and then to Windows. This was the first time that anyone had offered a

Component Software Development: An Introduction

standard for creating and using software components, and a huge third-party market emerged. VBXs evolved into OLE Control Extensions, and then into ActiveX Controls, which offered Web compatibility.

As the same time, Microsoft began to build server support into the COM model and introduced a distributed COM architecture (DCOM) into Windows NT 4 by the late 1990s. The newest version, COM+, works with Windows 2000 and offers transactions, publish-and-subscribe event handling, and asynchronous messaging.

Today, COM provides several variations that work with servers as well as clients. The model supports in-process servers (ActiveX controls), local servers (COM), and remote servers (DCOM); includes an interface definition language (Microsoft IDL) for creating COM interfaces; and provides RPC messaging support.

Sun Microsystems JavaBeans and EJB

Sun's JavaBeans and Enterprise JavaBeans provide a component model that sits on top of an object-oriented application architecture. The idea was to define a container for packaging related Java objects into reusable, platform-independent components that could that work on any device with a compatible JVM.

The original JavaBean specification supported GUI-based components for use on Java clients. A year later, Sun extended the JavaBean model to nonvisual, server-side components and introduced these as Enterprise JavaBeans. The EJB specification eliminated the GUI requirement and extended the component model to communicate with back-end applications and servers.

Initially only a few vendors offered JavaBeans, primarily because Microsoft already controlled the client-side component market with ActiveX controls. On the other hand, third-party vendors flocked to produce EJB-compliant software, and a huge market emerged not only for components, but also for servers that used EJBs to integrate heterogeneous applications across the Web. Many of these server vendors offer their own EJB containers designed to work with their products.

Currently, beans can add functionality to applications, applets, and servlets. They can communicate with each other, persist across different sessions, and hold state information if needed. The container provides the interface to the outside world and, as such, manages how each bean interacts with other beans, applications, and components. There are also bridging technologies that let JavaBeans and EJBs interoperate with COM components on Windows platforms.

OMG CORBAcomponents

OMG's CORBA architecture provides a well-defined set of services that allow object-oriented applications to communicate with one another. CORBA defines a message broker, or ORB, that routes messages and replies between objects irrespective of their geographic location or the language in which they were created. It also includes its own interface definition language, CORBA IDL, for describing object interfaces and translating data from one format to another and uses TCP/IP as its message transport protocol.

Although CORBA is still primarily an object-oriented architecture, the OMG is working to adopt a CORBA component model. CORBAcomponents— a superset of EJB that supports a variety of programming languages— specifies how to combine objects into components and describes how these components should behave within the CORBA architecture.

The model includes a container that provides multiple interfaces, event handling, persistence, transactions, and security. It will also include installation and distribution features, including XML configuration.

Component Software Development: An Introduction

The idea is to promote reuse by providing a larger-grained software unit that would work with heterogeneous applications. It will also provide a true component model for C++, which is the most common language for CORBA-compliant systems, and should appeal to application server as well as ORB vendors.

Technology Analysis

During the past two years, vendors and developers have become increasingly interested in components. Much of the effort expended in updating products or creating new applications has begun to drift toward components rather than objects.

Objects work well at the analysis and design stage for visualizing people, systems, and processes and mapping out their interactions. They tend to be much too small-grained— that is, they operationalize very specific features within a specific environment— to be reused, even at the design level.

Also, their reliance on inheriting methods and attributes from parent classes makes it harder to reuse them in different environments or combine them into larger units, since developers may have to drag along an entire hierarchy just to reuse a single class. (JavaBeans and EJBs, however, have overcome this limitation by strengthening the interface and letting the container manage behavior).

Taligent— the now-disbanded alliance between Apple and IBM— ran into these difficulties with CommonPoint, which was intended to be a completely object-oriented application framework. Taligent developers were aware of the granularity problem, so they overdesigned their objects for maximum flexibility. The objects had to expose much of their detail, making the framework extremely complex. What resulted were large, monolithic applications rather than compositional applications based on plug-and-play technology. CommonPoint was also built in C++, which does not directly support components. The project eventually died in the mid-1990s.

IBM took many of these lessons to heart when it began to build the San Francisco application framework in 1996. Instead of using a structured language with object extensions, the developers built the framework in Java, a fully object oriented language. They also created their own component model, San Francisco Beans, for the project, since EJBs were at the experimental stage. Even though San Francisco is complex and not especially easy to use, its component-based architecture allowed IBM to complete— rather than disband— the project. More recently, smaller-scale EJB component frameworks like The Theory Center have been very successful.

GUI frameworks, communications and storage libraries, and commercial components— albeit for the COM model— were available long before the JavaBeans craze. Although these required a lot of customization, they were still easier to use than building the functionality from scratch. Even packaged application vendors like Baan and SAP support component-based development, and they are starting to offer technology upgrades as plug-ins to existing environments.

True plug-and-play components, however, are still scarce due to economics. Reusability requires a long-term commitment, the components often cost five times as much as nonreusable software, and it normally takes years to amortize a component over multiple projects. Most projects require some type of server or message broker to translate and route information, which also drives up the cost.

Today, many organizations are turning to shrink-wrapped and internally built components, especially for volatile e-business and Web commerce applications. Most firms use components and APIs to add functionality to existing applications, although there are some bleeding-edge organizations creating entire applications using components.

Component Software Development: An Introduction

One of the more interesting applications for components is a plug-and-play operating system. OS/2, Unix, and the Mac OS have used this technology with varying degrees of success, but the most promising applications is for consumer and embedded devices. Developers would be able to select a kernel and then drag and drop functionality from component libraries. These components would make it easy to build custom operating systems for specific platforms as well as add new features if needed. Sun's Jini uses this type of technology to deliver services to consumer devices using a networked operating system.

Business Use

As software systems become much larger and more complex, many organizations have begun to incorporate components to upgrade existing applications or create new ones. Some organizations are purchasing prebuilt components— especially EJBs and ActiveX controls, others are creating their own, and still others are combining the two approaches. As components evolve, they are beginning to provide their own infrastructure services or automatically map to existing structures. This will help leverage the skills of business programmers and advanced users.

Well-documented COM and JavaBean standards— as well as the need to quickly create flexible, Web-based e-commerce systems— have fostered a large market for ready-to-use plug-and-play components as well as several commercial component frameworks. Although today's components are primarily horizontal, a market for industry- or function-specific vertical components is taking shape.

The market began with Microsoft's COM standard. Commercial COM, VBX, and ActiveX components have been available since Visual Basic and C++ became popular in the mid-1990s, and packaged application/ERP vendors have been offering COM plug-ins for several years.

Java components are also popular, especially on the server. Although client-side JavaBeans have not attracted much commercial interest, the EJB market has exploded, with application server, B2B/B2C, portal, packaged application, and IDE vendors all offering prebuilt components.

Benefits and Risks

Using components for creating and maintaining software applications has a number of benefits. These include:

- **Making it easy to integrate legacy applications with e-business and Web software.** This may include building a GUI for using or maintaining the legacy product, creating an API so the application works with brokering, or wrapping the entire application as an EJB or COM component and plugging it in to a present-day application.
- **Code reduction.** Instead of writing custom functionality, developers can call a component when needed. In addition, components— unlike objects— do not require complex inheritance hierarchies to be ported between different applications.
- **Incremental change.** Developers can add, remove, or change functionality gradually rather than via an expensive, time-consuming upgrade. This is especially important in multiple-tier applications where business processes may change faster than the application as a whole.
- **Easier management.** Administrators can look at or work with individual components without bringing down the entire system. On the other hand, it may be difficult to isolate one instance of a component in a distributed architecture with multiple copies.
- **Improved scalability.** Many distributed applications slow down because of limited database or transaction monitor connections. Vendors are beginning to offer components that cache server-side transaction or database logic to increase the number of connections and provide faster throughput.

Component Software Development: An Introduction

- **Excellent development tools.** Objects have been available for over 30 years, and today there is no shortage of tools for creating objects in UML, C++, Java, Smalltalk, or other languages. They range from "straight" object-oriented analysis and design tools to standard IDEs to still others that automatically generate code from UML or other models. Prices also range from free to hundreds of thousands of U.S. dollars.

Most of the risks associated with components are typical of newer technologies. These are likely to become less onerous as component specifications evolve.

- **Component interoperability.** It is difficult to mix components based on different architectures. EJB, CORBA, and COM require translation and messaging software to work together, even in different applications. In some cases, components using the same architecture may be problematic, such as EJBs that lose portability when tied to a specific application server. There are relatively few vendors who make integration software, and experts who know how to develop or configure these products are also scarce.
- **Component interactions.** It is relatively easy to create individual components and wire them together, but there are no tools for examining how they interact as subgroups. In a three-component system, the components may interact individually, in groups of two components, or as a single group of three components. How do the different groupings work with applications and affect performance? The combinatorics are staggering, and there is currently no way to account for these anywhere in the development life cycle.
- **Programming for reuse.** Creating genuinely reusable components requires time and money.
- **Pricing and licensing models:** These are evolving over time with customer input. This is even further complicated by an international software market.
- **Legal issues.** Intellectual property, ownership, and royalty issues are just beginning to be discussed. These are also subject to internationalization.
- **Upgrading and testing difficulties.** These are especially tricky if a component's source code is not available.

Standards

Three of the four component models described here have documented, well-defined standards. The fourth, CORBAcomponents, has not yet been ratified by the OMG membership. The table below identifies current standards and specifications.

Table 1: Component Standards

Name	Current Standards	Standardizing Organization
Common Object Model	COM ActiveX Controls DCOM COM+	Microsoft
CORBAcomponents	Currently in proposal stage, upcoming in CORBA 3.0	OMG
JavaBeans	JDK 1.2 Runtime Containment and Services Protocol 1.0 JavaBeans Activation	Sun Microsystems

Component Software Development: An Introduction

Name	Current Standards	Standardizing Organization
	Framework 1.0.1 Beans Development Kit 1.1	
Enterprise JavaBeans	EJB 1.1 J2EE	Sun Microsystems

Selection Guidelines

There are a few issues to address when selecting a product for developing or using components:

- **EJB:** The current standard is 1.1, but many products are compatible only up to EJB 1.0. The new specification now covers entity beans, state information, and persistence. Ask the product's vendor about which EJB version(s) are supported.
- **DCOM/COM+:** COM+ requires Windows 2000. Older development tools or servers may run on Windows 2000 but may not necessarily support COM+ features. Also, Windows 95 and Windows NT versions earlier than 4.0 may not be able to fully run DCOM components.

Technology Leaders

Over the past several years, new commercial components and a services market have emerged to complement— as well as challenge— established development tool and integration vendors. Many of these component vendors are start-ups, although some long-lived organizations are beginning to provide plug-and-play tools and functions.

ComponentSource

In existence since 1995, this online vendor distributes software components for horizontal and vertical markets around the world. It offers thousands of components in a variety of formats, including COM, Inprise/Borland Delphi, EJB, JavaBeans, JavaServer Pages, ActiveX, CORBA, Visual Basic, and C++. The customer base consists primarily of large corporations and there is also a matching service for component creators and component users.

EC Cubed

EC Cubed's ecWorks is a suite of e-commerce application components for developing electronic commerce applications. The suite includes data integration, security, workflow management, B2B, and event notification components. Customers can combine any of all of the components into full-scale e-commerce applications

Flashline.com

Flashline.com's online store offers JavaBeans and EJBs to professional developers and Global 2000 companies. It markets prebuilt shrink-wrapped products from third-party vendors, components originating with internal corporate development projects, and even components designed for U.S. Government agencies. The vendor provides a registry database of available components as well as a service that matches developers with organizations that need custom components.

IBM

IBM's San Francisco is a framework for component development in Java. It includes components for developing an application infrastructure and horizontal business processes, as well as ledger, warehouse management, order management, and accounts receivable/payable frameworks. When the product was introduced in 1996, IBM had to create its own Java component model— San Francisco Beans— since

Component Software Development: An Introduction

there were no other Java component standards. IBM is transitioning to support Sun's EJB model in upcoming releases and will help existing San Francisco customers migrate their applications. Customers are mostly ISVs developing commercial software.

ILOG

ILOG has been selling high-end optimization, data integration, and visualization components since 1987. Today, this French company offers libraries in both C++ and Java for Windows, Unix, and Linux platforms. The vendor's components are used in telecommunications, manufacturing, transportation, government, defense, and B2B applications.

Rogue Wave Software

Rogue Wave has been shipping software components since it introduced its first C++ class library in 1989. It was one of the few vendors successfully marketing prebuilt C++ and COM software and continues to offer infrastructure, analytical, database GUI, and mainframe connectivity libraries. Rogue Wave also supplies Java, CORBA, and enterprise integration components.

The Theory Center/BEA Systems

The Theory Center—acquired by BEA Systems in November 1999—markets prebuilt commercial EJBs for electronic business. It offers over 80 components for CRM, online order management, updateable catalogs, trouble ticket management, and workflow management applications. The components can be used to extend existing applications or glued together to create new software systems.

Datapro Insight

Today, components seem to be winning the battle for code reuse, and have overtaken objects as the preferred construct for developing flexible software systems. Components actually function as black-box software, offering well-designed interfaces that encapsulate behavior and can be used in different environments. Components are also much larger-grained than objects, so they can incorporate a larger range of functions. Although a strong market for prebuilt Microsoft COM components has existed for nearly a decade, Sun's EJB model is becoming extremely popular, especially for Web-enabled applications running on heterogeneous platforms. One of the most difficult problems with component-based systems is how to manage interactions between groups of two or more components and other parts of the system.